

Table 的滚动偏啦

Table 的滚动偏啦

最近我们遇到了一个 Table 横向滚动错位的问题。在虚拟滚动场景下，用户拖动横向滚动条，表体已经滚动到正确位置，但是表头偶尔会停在旧的位置。尤其是在滚回最左侧时，表体已经是 0，表头却停在中间位置。

图片说明：Table 横向滚动错位

从现象看，像是 Table 的多个滚动区域没有同步好。由于 Table 在固定表头、固定列、汇总行、sticky 滚动条之间都有滚动联动逻辑，所以这种问题排查起来会比较绕。本文记录一下这次排查过程。

先看滚动同步

Table 横向滚动时，并不只有一个 DOM 在滚动。固定表头、表体、汇总行、sticky 滚动条都维护着自己的横向位置。因而任意一个区域触发 onScroll 后，都会进入同一个同步流程：

```
// Simplified Code
function onInternalScroll({ currentTarget, scrollLeft }) {
  const mergedScrollLeft = typeof scrollLeft === 'number' ? scrollLeft :
currentTarget.scrollLeft;

  // Sync other scroll areas
}
```

在这个流程里，当前拿到的 scrollLeft 会被同步给其它区域：

```
forceScroll(mergedScrollLeft, scrollHeaderRef.current);
forceScroll(mergedScrollLeft, scrollBodyRef.current);
forceScroll(mergedScrollLeft, scrollSummaryRef.current);
forceScroll(mergedScrollLeft, stickyRef.current?.setScrollLeft);
```

而 forceScroll 的作用也很直接，就是把目标元素设置到同一个横向位置：

```
// Simplified Code
function forceScroll(scrollLeft, target) {
  if (typeof target === 'function') {
```

```
target(scrollLeft);
} else if (target.scrollLeft !== scrollLeft) {
  target.scrollLeft = scrollLeft;
}
}
```

这里有一个容易忽略的点：onScroll 的来源并不等同于“用户正在操作的区域”，它只是事件最先冒出来的那个元素。通过代码设置 scrollLeft 也可能触发该元素自己的 scroll 事件。也就是说，表体滚动后同步表头，表头也可能再触发一次 onScroll。

因而第一反应是：是不是表头和表体在抢滚动来源？

看起来像锁

rc-table 内部有一个短暂的滚动锁，用于避免同步滚动时互相触发事件。简化管理，它会记录当前正在滚动的元素，然后在一小段时间内忽略其它同步产生的滚动事件。

这个逻辑本身是合理的。如果没有这个锁，那么表体滚动会同步表头，表头滚动又同步表体，就可能出现循环触发。

图片说明：原始滚动锁流程

但是在这个 issue 里，它看起来也很可疑。因为用户实际拖动的是表体，但是同步表头时，表头的 scroll 事件也会触发。如果表头先抢到了锁，那么后续表体的真实滚动就可能被忽略。

于是我们先尝试了一个方向：记录当前鼠标 hover 在哪个滚动区域上。用户鼠标所在的表体、表头、汇总行或 sticky 滚动条，才是真正应该驱动同步的来源。

这个方案看起来可以解释问题：

- 用户拖动表体时，表体应该优先；
- 同步触发的表头 scroll 不应该反过来抢锁；
- sticky 滚动条也可以用同样的来源判断。

图片说明：基于 Hover 的滚动同步

但是很快，我们发现这个方向并不是根因。

日志对不上

为了确认到底是谁在触发滚动，我们直接在 antd 的 node_modules 里给 rc-table 打日志。日志中记录当前触发源、锁定源、输入的 scrollLeft，以及同步到目标元素的值。

图片说明：Table 滚动同步日志

关键日志大致如下：

```
onInternalScroll { source: 'body', inputScrollLeft: 794.171875 }
onInternalScroll { source: 'body', inputScrollLeft: 688.171875 }
onInternalScroll { source: 'body', inputScrollLeft: 375.171875 }
onInternalScroll { source: 'body', inputScrollLeft: 55.171875 }
onInternalScroll { source: 'body', inputScrollLeft: 0 }
```

从这里看，表体的滚动事件是正常上报的。更重要的是，0 也已经进入了同步流程。也就是说，并不是表体滚回 0 时被锁挡住了。

但是后面出现了另一组日志：

```
retry sync target element { targetName: 'header', from: 0, to: 794.171875 }
retry sync target element { targetName: 'header', from: 794, to: 688.171875 }
retry sync target element { targetName: 'header', from: 688, to: 375.171875 }
retry sync target element { targetName: 'header', from: 375, to: 55.171875 }
```

这就很奇怪了。既然 0 已经同步过了，为什么表头还会被写回旧值？

于是问题从“谁抢了滚动来源”，变成了“谁在后面把旧值写回去了”。

延迟同步

回头看 forceScroll，会发现它并不只是简单设置一次 scrollLeft。真实逻辑里还存在一个延迟重试：

```
function forceScroll(scrollLeft, target) {
  if (typeof target === 'function') {
    target(scrollLeft);
  } else if (target.scrollLeft !== scrollLeft) {
    target.scrollLeft = scrollLeft;

    // Delay to force scroll position if not sync
    if (target.scrollLeft !== scrollLeft) {
      setTimeout(() => {
        target.scrollLeft = scrollLeft;
      }, 0);
    }
  }
}
```

这段逻辑是有历史原因的。在动态列、布局变化等场景里，部分浏览器设置 `scrollLeft` 后，目标元素可能没有停在预期位置。因而代码会在设置后立刻读一次，如果发现 `target.scrollLeft !== scrollLeft`，就排一个 `setTimeout`，在下一轮事件循环里再写一次。

而这次问题里，表体连续上报了多个横向位置：

```
794.171875
688.171875
375.171875
55.171875
```

单独看这个逻辑没问题。但是在连续滚动里，它就会产生旧值回写：

1. 表体滚动到 794.171875；
2. 表头被设置为 794.171875；
3. 代码发现表头当前位置仍不等于这次期望值，于是排了一个写回 794.171875 的 `timeout`；
4. 用户继续拖动，表体滚回 0；
5. 表头被同步为 0；
6. 这次写入后立即读到的值已经是 0，旧逻辑不会再排一个确认 0 的 `timeout`；
7. 前面那些旧 `timeout` 执行，把表头又写回旧值。

图片说明：时间线示意图，展示 794.171875/688.171875/375.171875/55.171875 的 `timeout` 排队在 0 同步之后执行

至此，问题就很清楚了。真正导致偏移的不是滚动锁，而是延迟重试里闭包保存的旧 `scrollLeft`。

修复

最直接的方式，是让每个目标元素只保留最新的一次重试。

换句话说，旧的 `timeout` 本来就是为了兜底“这一次同步没有生效”。但是一旦同一个目标元素收到了新的同步，旧 `timeout` 就已经失去了意义。此时应该先把旧 `timeout` 取消掉，再根据最新的 `scrollLeft` 判断是否需要排新的 `timeout`：

```
// Simplified Code. See rc-table for the complete implementation.
const [scrollRetryTimeoutMap] = React.useState(() => new WeakMap());

function forceScroll(scrollLeft, target) {
  clearTimeout(scrollRetryTimeoutMap.get(target));
```

```
if (target.scrollLeft !== scrollLeft) {
  target.scrollLeft = scrollLeft;

  // Delay to force scroll position if not sync
  const retryTimeout = setTimeout(() => {
    if (target.scrollLeft !== scrollLeft) {
      target.scrollLeft = scrollLeft;
    }
  }, 0);

  scrollRetryTimeoutMap.set(target, retryTimeout);
}
}
```

这里用 WeakMap 记录每个 DOM 元素对应的 retry timer。它的好处是，DOM 元素作为 key，不需要我们额外做生命周期清理。当表头从 794.171875 又同步到 0 时，0 这次同步会先把前面排队的旧 timeout 取消掉。于是旧值没有机会在最后回写，表头就能停在最新位置。

总结

这次问题有趣的地方在于，最开始的猜测并不是完全错误。同步 scrollLeft 确实会触发滚动事件，滚动锁也确实会影响滚动同步。但是最终日志证明，问题并不在滚动来源，而在延迟任务里保存了旧值。

在排查类似问题时，如果某个状态“明明已经设置对了，但是稍后又变错了”，就很值得检查是否存在延迟任务、动画帧、Promise 或者其它异步回调。因为最终落到 DOM 上的，往往不是最正确的一次写入，而是最后一次写入。

以上。